Evaluation of CNN based Minimax Chess Model

Colby Daniel, Brailey Sharpe, Matthew Shirley

CS 3270: Intelligent Systems

Dr. Md Shirajum Munir

April 27th, 2026

Introduction

This project seeks to utilize the Chess Evaluations dataset, created with stockfish evaluations set at a depth of 22. The dataset is converted from FEN notation to a chess board, then the chess boards are converted to a 12x8x8 tensor. This tensor is then fed into the CNN model where it uses 19 layers of information to determine a single evaluation for any given position. This analysis will first be a comparison of the models' training loss.

Architecture

This project features a CNN enhanced with residual blocks to evaluate chess positions. This mode's objective is to approximate the value function used by modern chess engines given a board. Each board is encoded as a 8x8 grid with 19 channels to capture all the information needed to draw up an accurate evaluation. Inspired by architectures used in AlphaZero and Stockfish-style engines. The model contains an initial 3x3 convolution layer with 128 filters to extract low-level spatial features. The core of the model consists of residual blocks each containing: a convolution, batch normalization, ReLu activation, and a skip connection. These blocks allow the network to learn deeper nuanced patterns of a board to better connect them to certain evaluations.

Training Results

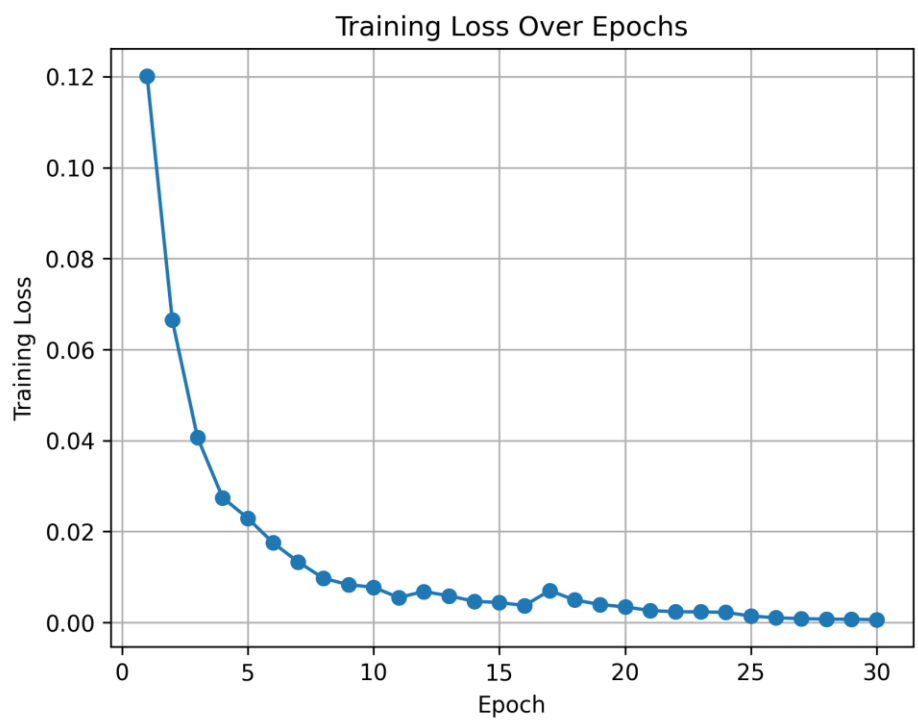
Configuration:

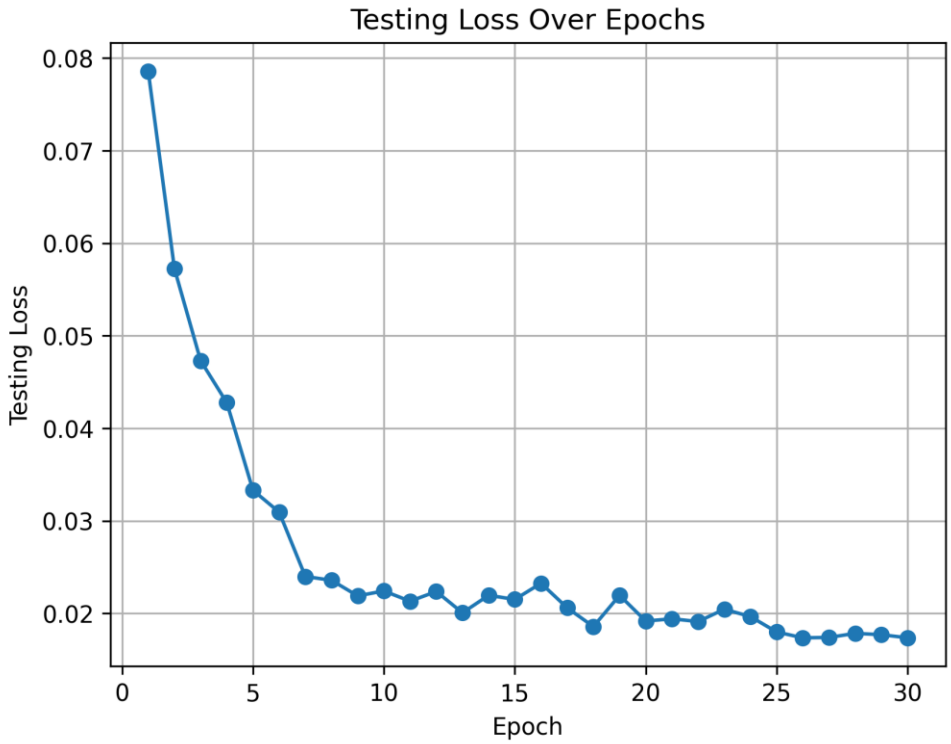
Loss Function: MSELoss

Optimizer: Adam

Model: evaluation_cnn_model2.pth

```
Train Loss: 0.0006
Test  Loss: 0.0174
MAE: 0.0816 | RMSE: 0.1324 | R2: 0.8697
----------------------------------------
Average Training Loss: 0.0133
Average Testing Loss: 0.0259
```




Testing Loss Over Epochs
Testing Loss
Epoch
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0
5
10
15
20
25
30

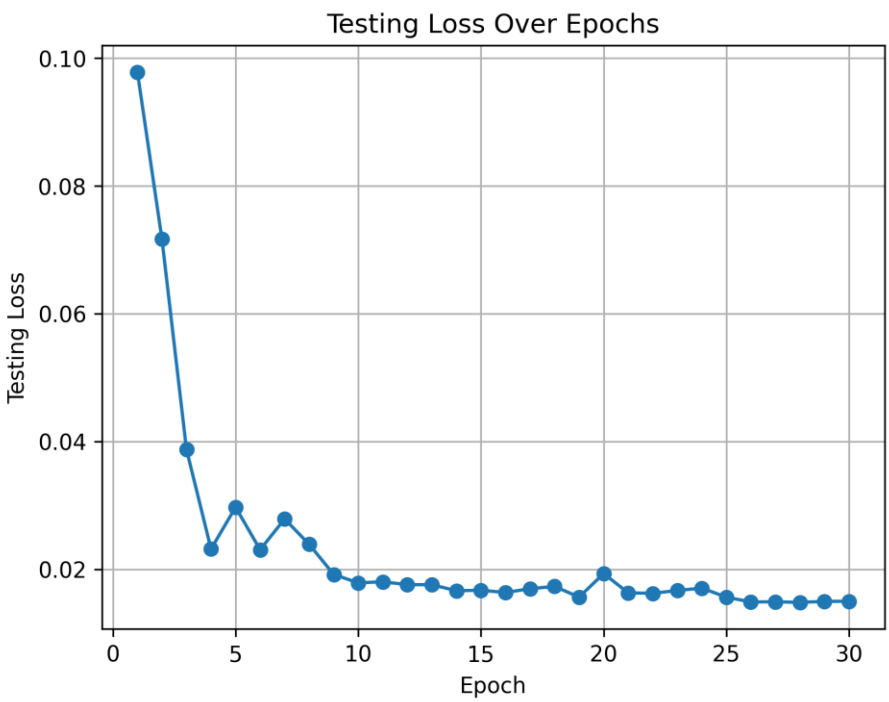
Configuration:

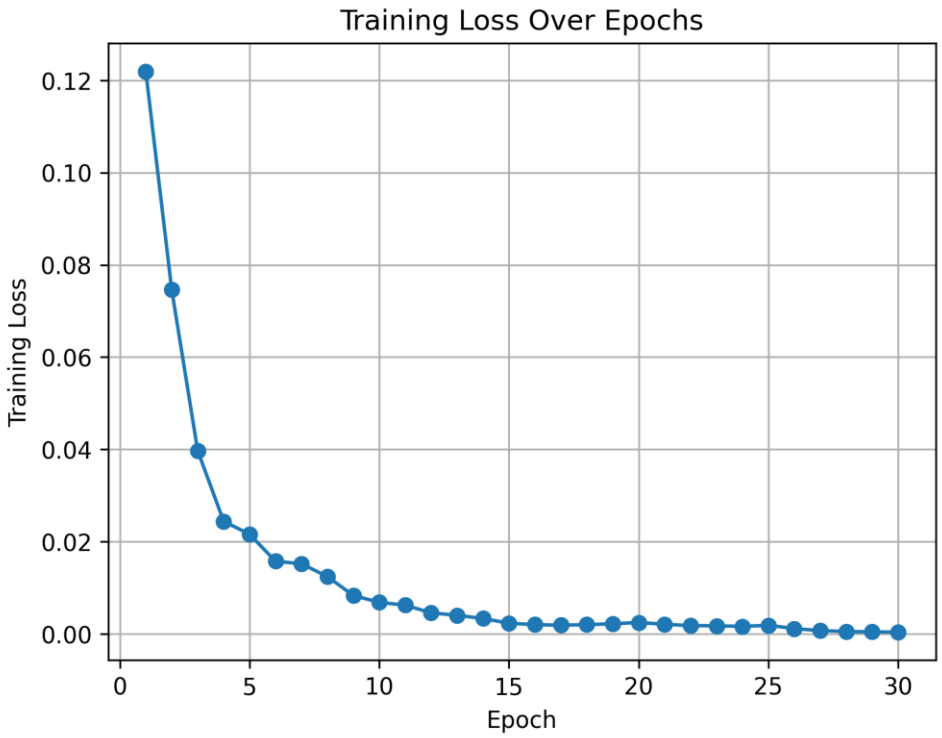
Loss function: MSELoss

Optimizer: AdamW

Model: evaluation_cnn_model3.pth

```
Epoch 30/30
Train Loss: 0.0004
Test  Loss: 0.0150
MAE: 0.0756 | RMSE: 0.1227 | R2: 0.8880
----------------------------------------
Average Training Loss: 0.0128
Average Testing Loss: 0.0234
```





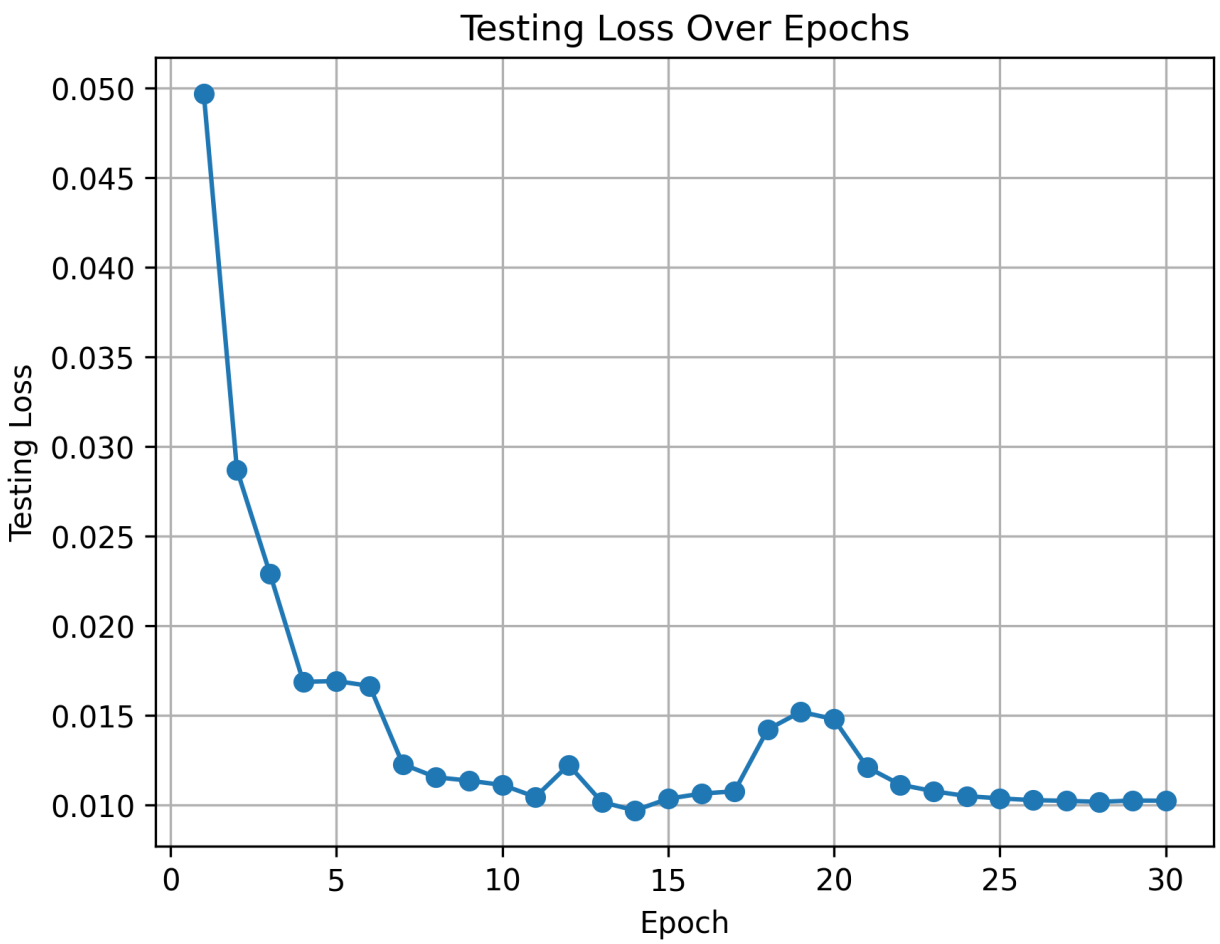
Configuration:

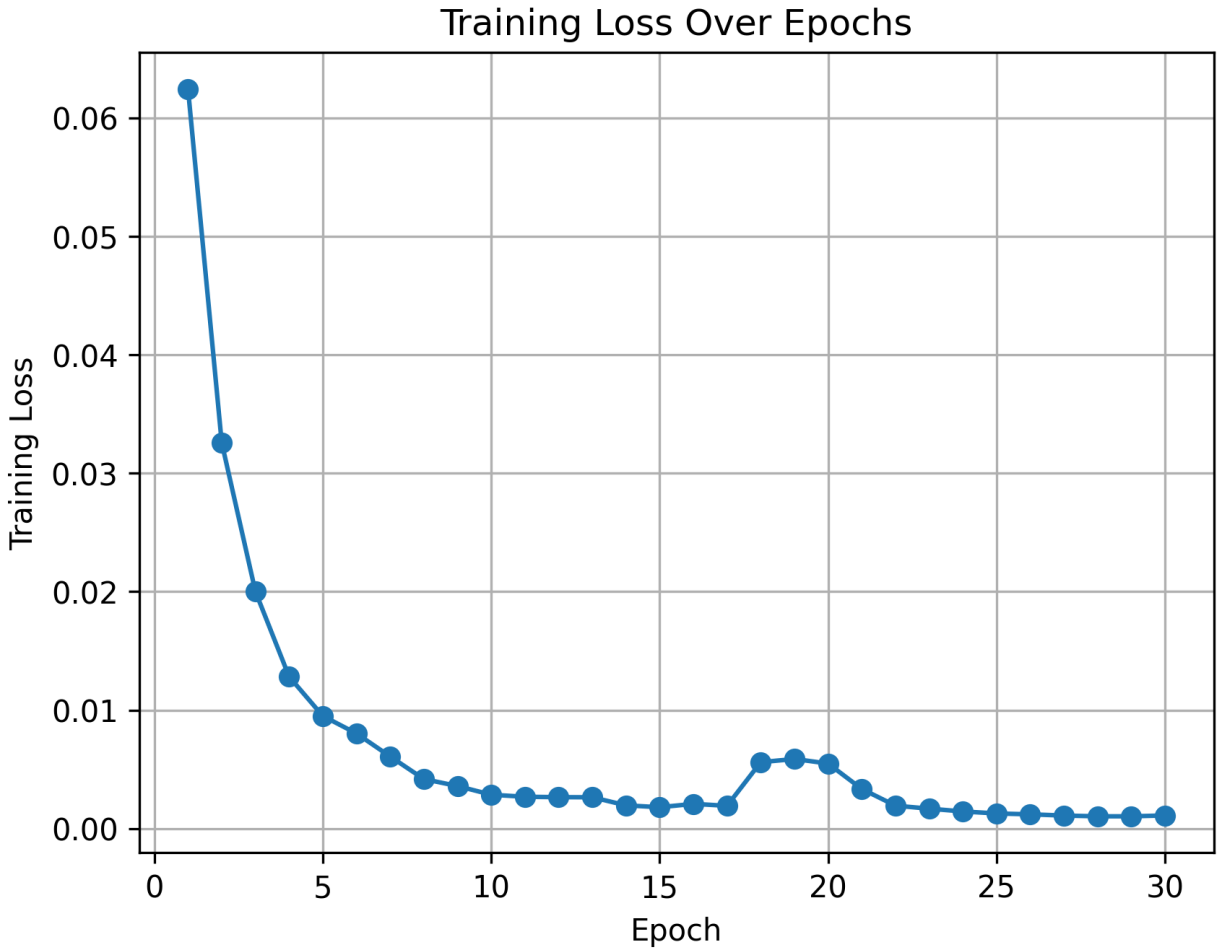
Loss function: SmoothL1Loss

Optimizer: Adam

Model: evaluation_cnn_model0.pth

```
Epoch 30/30
Train Loss: 0.0011
Test  Loss: 0.0102
MAE: 0.0867 | RMSE: 0.1435 | R2: 0.8470
----------------------------------------
Average Training Loss: 0.0070
Average Testing Loss: 0.0141
```


Testing Loss Over Epochs
Testing Loss
0.050
0.045
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0
5
10
15
20
25
30
Epoch



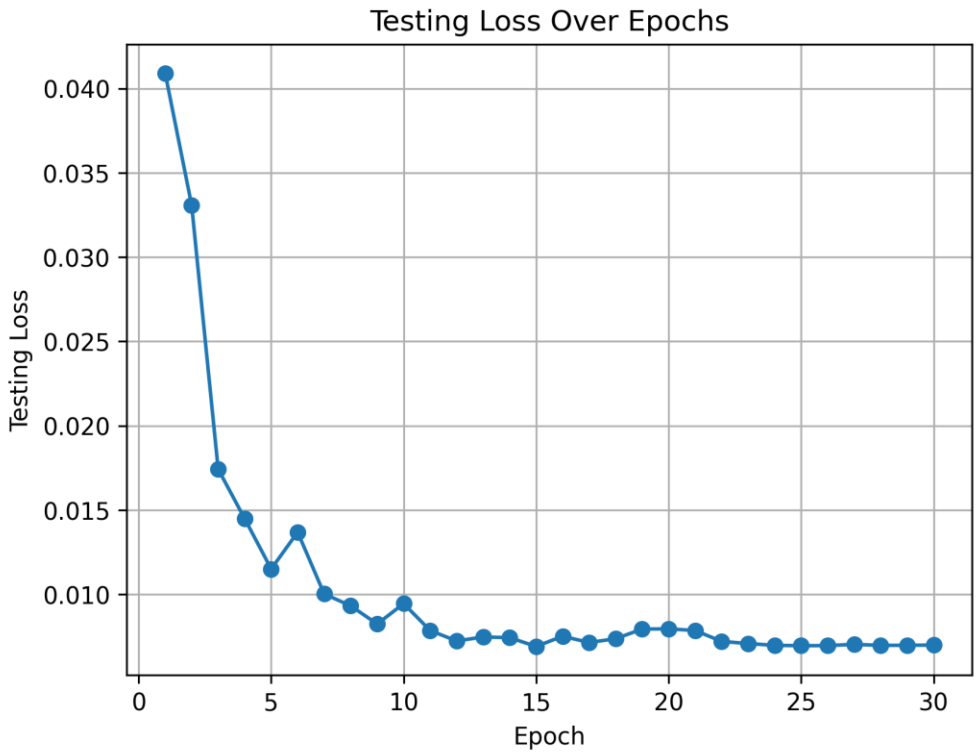
Configuration:

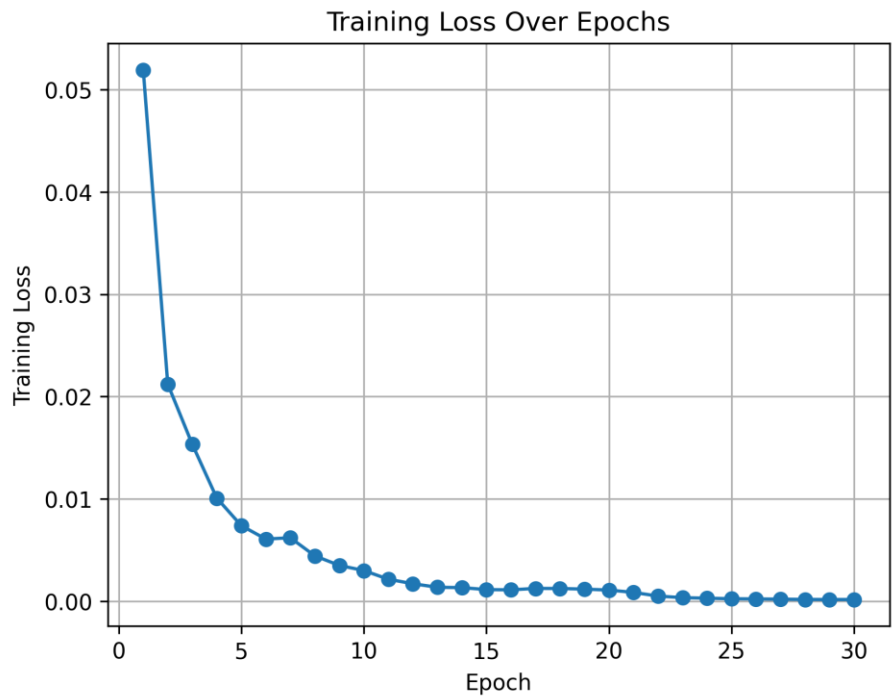
Loss function: SmoothL1Loss

Optimizer: AdamW

Model: evaluation_cnn_model1.pth

```
Epoch 30/30
Train Loss: 0.0002
Test  Loss: 0.0070
MAE: 0.0724 | RMSE: 0.1186 | R2: 0.8955
----------------------------------------
Average Training Loss: 0.0049
Average Testing Loss: 0.0105
```





Summary of Training

Most models performed relatively the same with our current CNN structure. The models tend to converge at around 10-15 epochs with our current dataset of 2000 items. Overall, the AdamW and SmoothL1Loss model performed the best based on the average loss and will be used in the next section of testing. This is likely due to AdamW being a more stable optimizer for our current CNN architecture. As for MSE vs smoothL1loss, MSE tends to punish the model more severely for outliers as it squares the error, while smoothL1 tends to act like MSE for small values, but transitions to a linear penalty with large values.

Stockfish Comparison

This section seeks to optimize the capabilities of the AI opponent using the model currently trained. This is done by comparing the minimax algorithm and negamax algorithms with heuristic based material-based functions. This will be done by making a move with the

model, then comparing its move against stockfish at various depths. As depth increases, the model will compare against better stockfish models. Overall, both minimax and negamax will be run once at each depth, giving a small indication as to how they compare against the baseline stockfish model known for its accuracy.

Configuration:

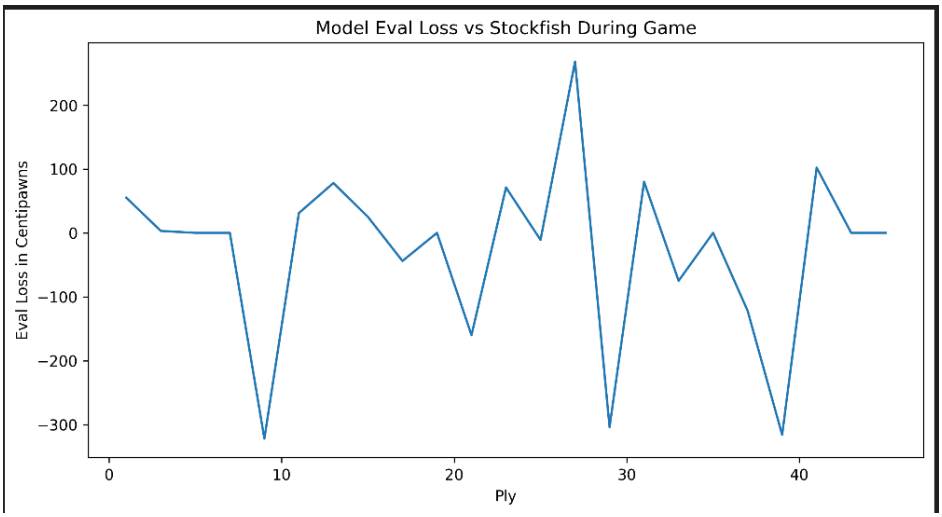
Loss Function: MSELoss

Optimizer: AdamW

Model: evaluation_cnn_model1.pth

Depth 1

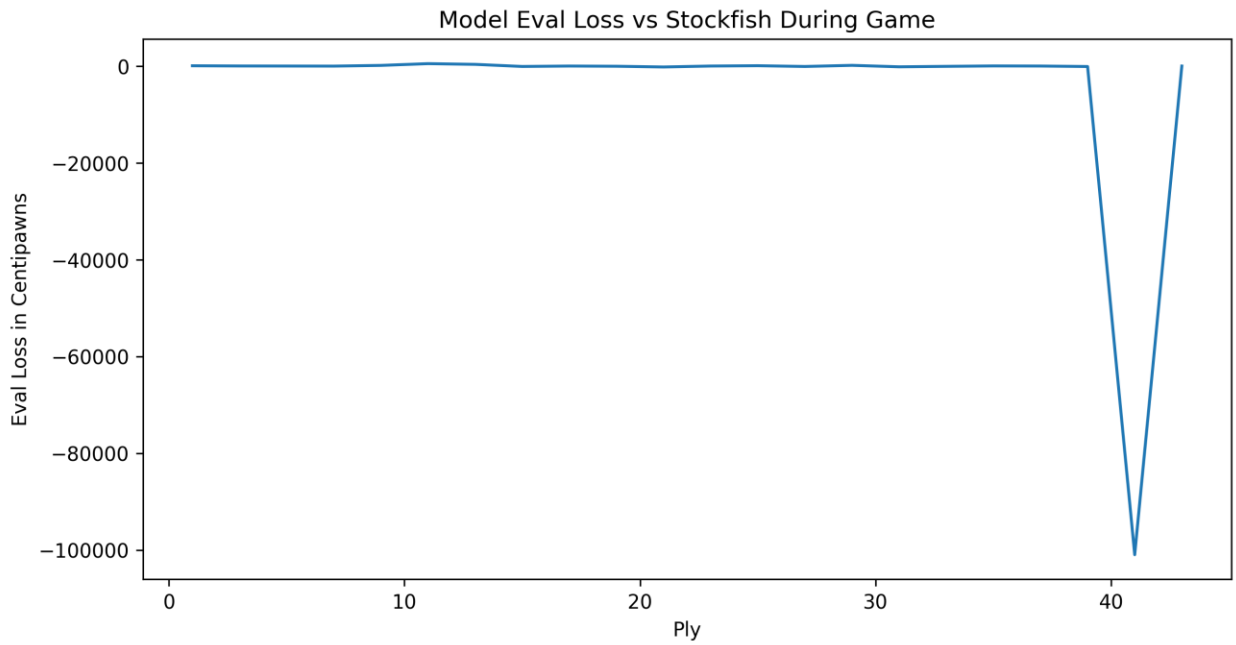
NegaMax




```
Final result: 1-0
Mean: 89.8695652173913
Median: 55.0
Trimmed mean: 68.04761904761905
90th percentile: 296.8
95th percentile: 314.79999999999995
Max: 322
Blunder rate >300cp: 0.13043478260869565
```

MiniMax

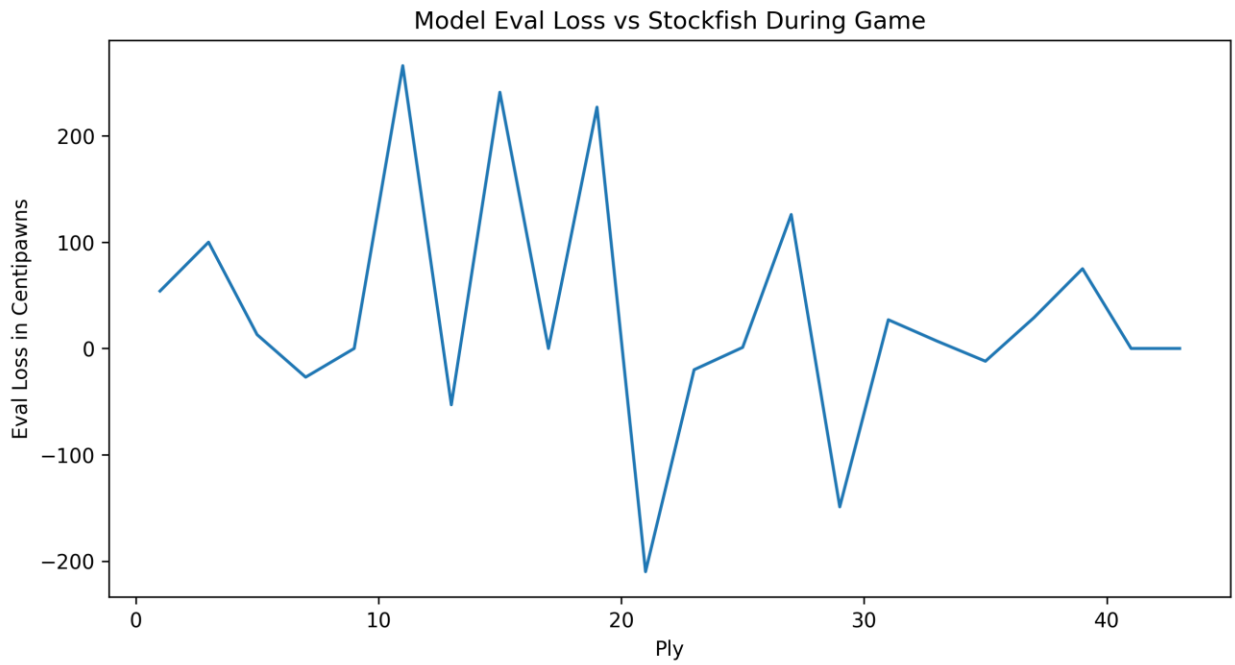
```
Final result: 1-0
Mean: 4682.090909090909
Median: 66.0
Trimmed mean: 76.95
90th percentile: 331.00000000000034
95th percentile: 484.6999999999999
Max: 100975
Blunder rate >300cp: 0.13636363636363635
```



Depth 2

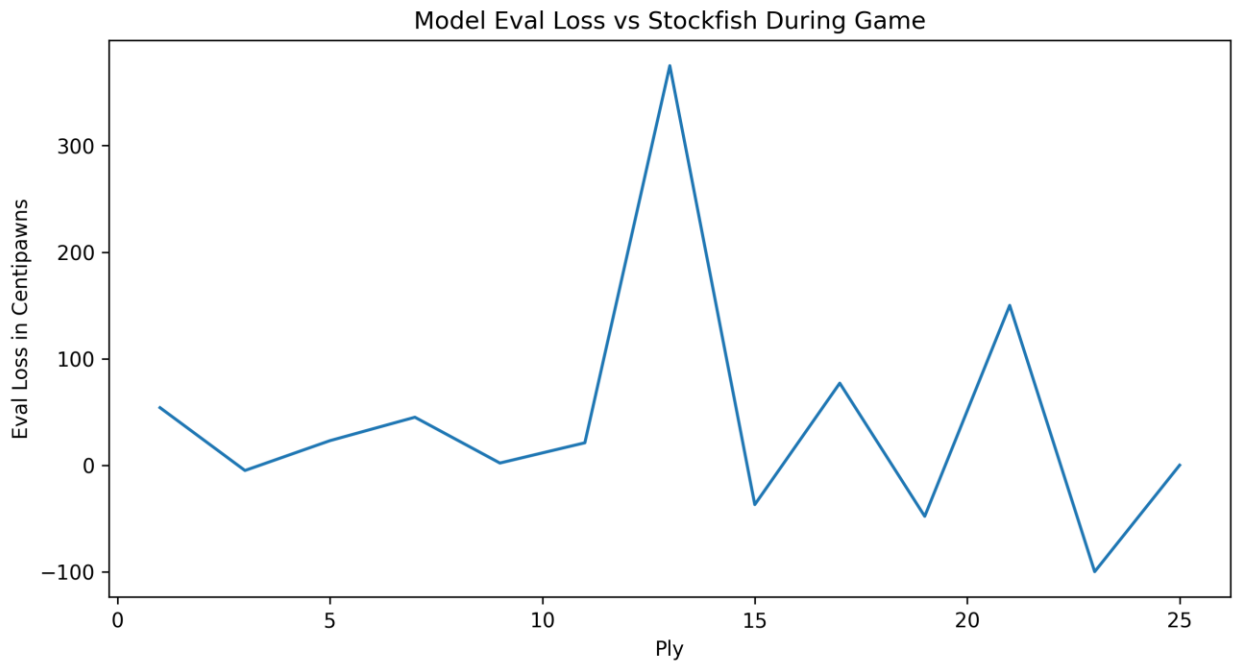
NegaMax

```
Final result: 1-0
Mean: 74.4090909090909
Median: 28.0
Trimmed mean: 56.5
90th percentile: 225.30000000000004
95th percentile: 240.29999999999998
Max: 266
Blunder rate >300cp: 0.0
```



Minimax

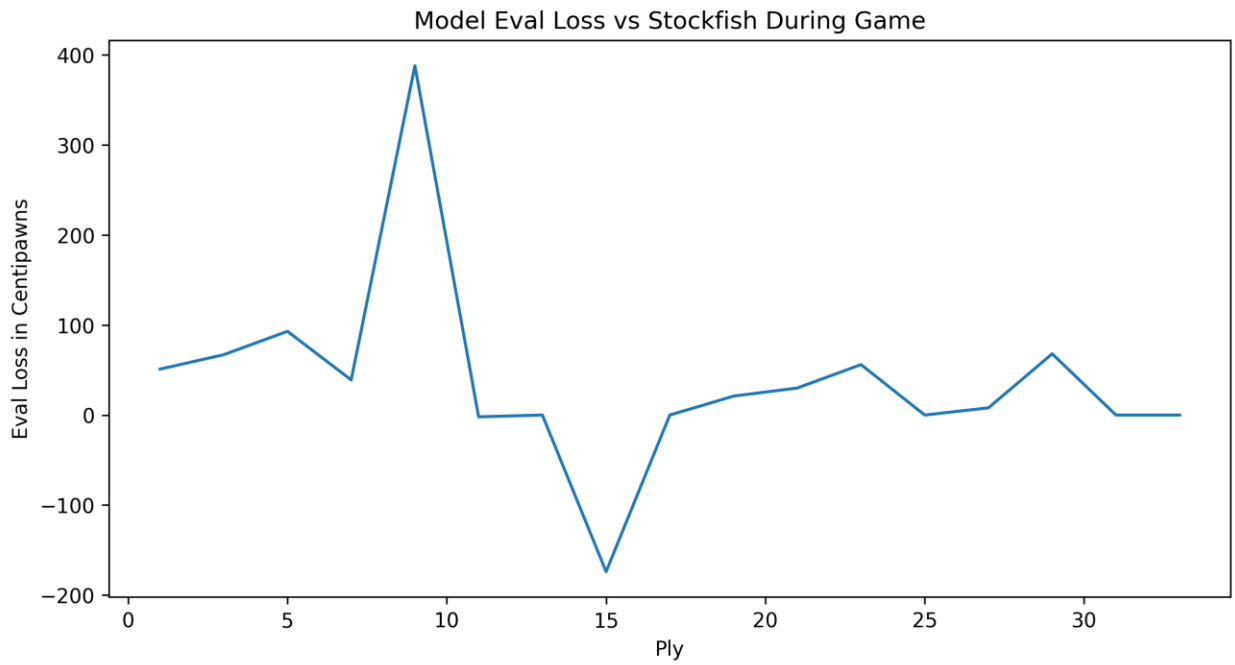
```
Final result: 1-0
Mean: 72.07692307692308
Median: 45.0
Trimmed mean: 46.833333333333336
90th percentile: 140.00000000000003
95th percentile: 239.9999999999997
Max: 375
Blunder rate >300cp: 0.07692307692307693
```



Depth 3

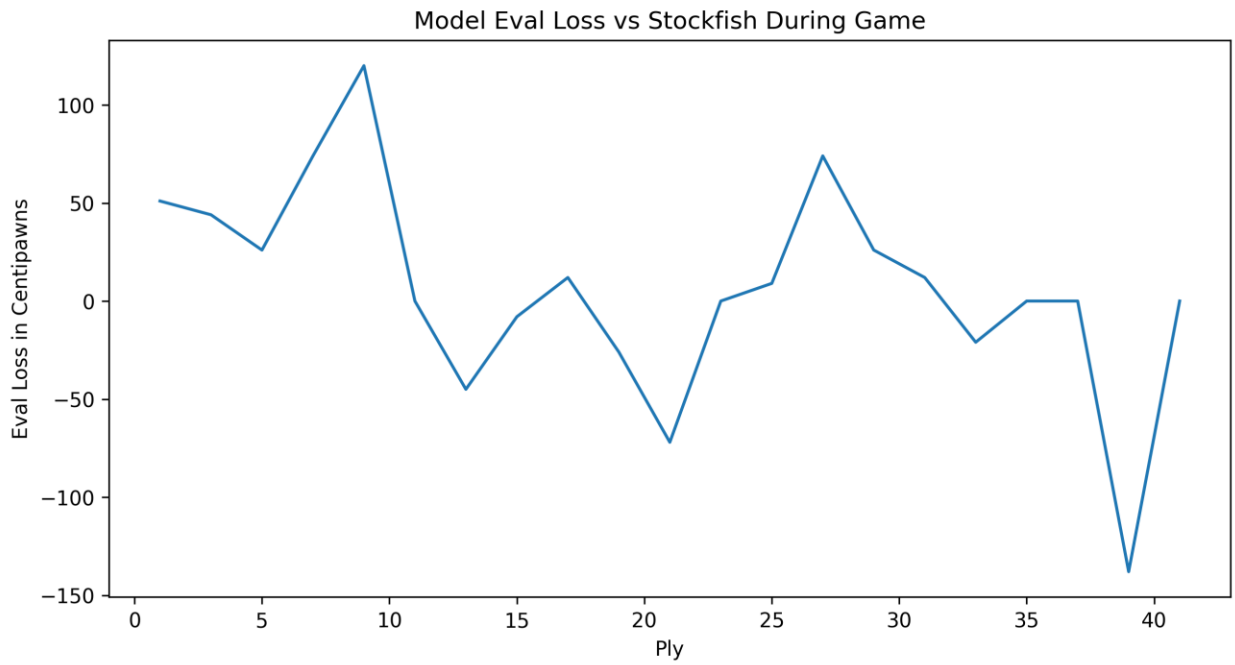
Negamax

```
Final result: 1-0
Mean: 48.25
Median: 59.0
Trimmed mean: 39.333333333333336
90th percentile: 72.6
95th percentile: 73.8
Max: 75
Blunder rate >300cp: 0.0
```



Minimax

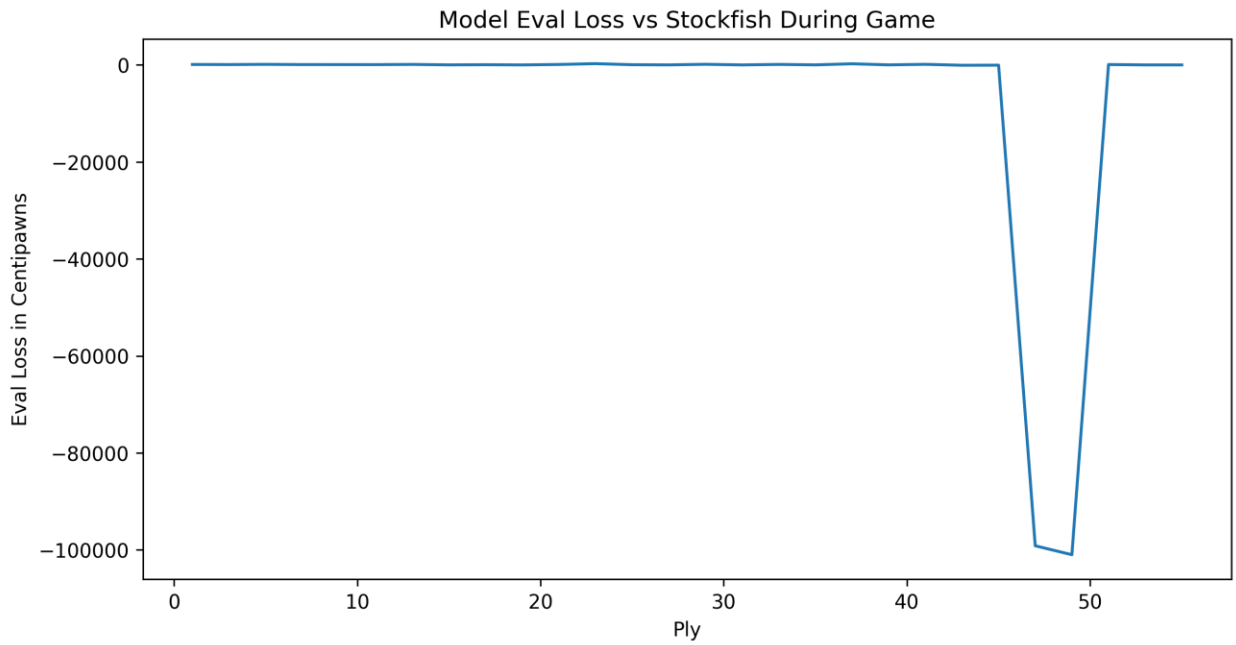
```
Final result: 1-0
Mean: 36.095238095238095
Median: 26.0
Trimmed mean: 31.0
90th percentile: 74.0
95th percentile: 120.0
Max: 138
Blunder rate >300cp: 0.0
```



Depth 4

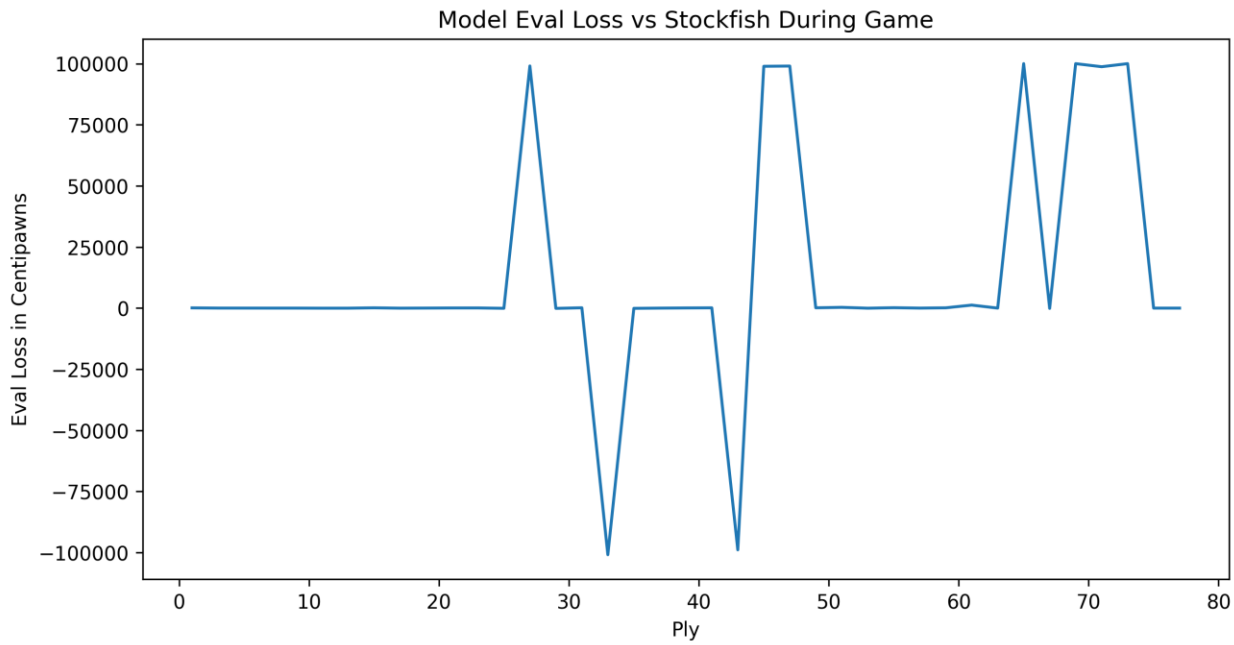
Negamax

```
Final result: 1-0
Mean: 7214.321428571428
Median: 70.5
Trimmed mean: 70.84615384615384
90th percentile: 241.10000000000002
95th percentile: 64545.649999999856
Max: 100998
Blunder rate >300cp: 0.07142857142857142
```



Minimax

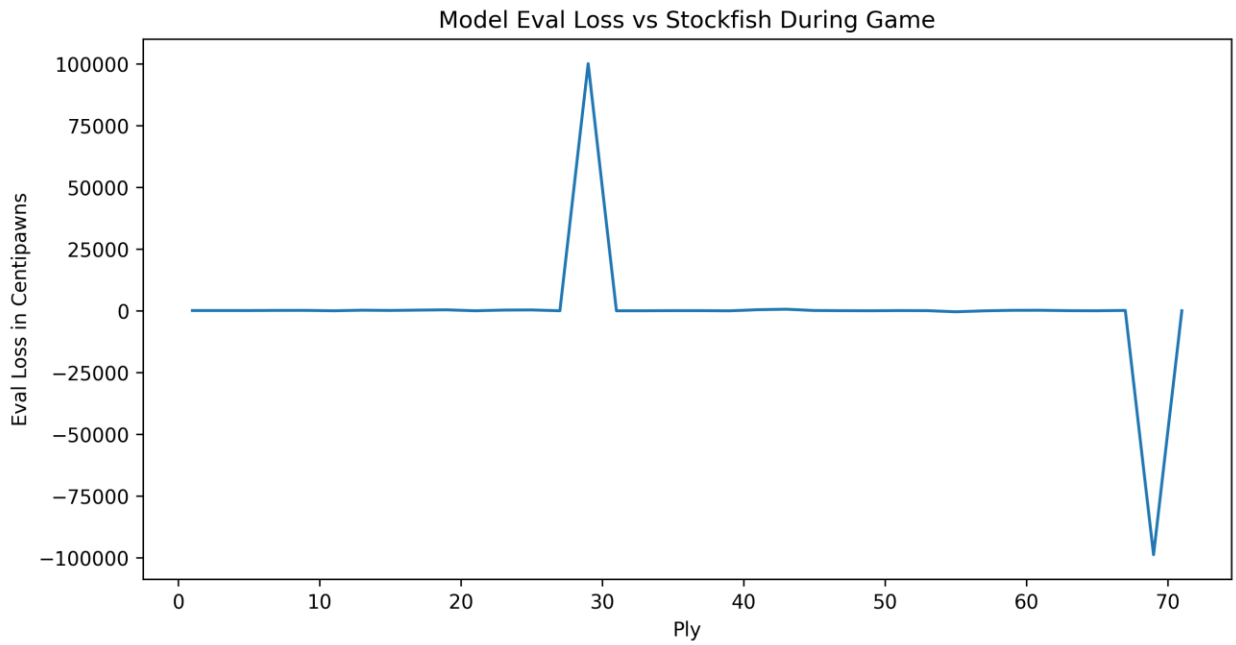
```
Final result: 1-0
Mean: 23042.23076923077
Median: 93.0
Trimmed mean: 20994.13157894737
90th percentile: 99247.2
95th percentile: 100000.0
Max: 100870
Blunder rate >300cp: 0.28205128205128205
```



Depth 5

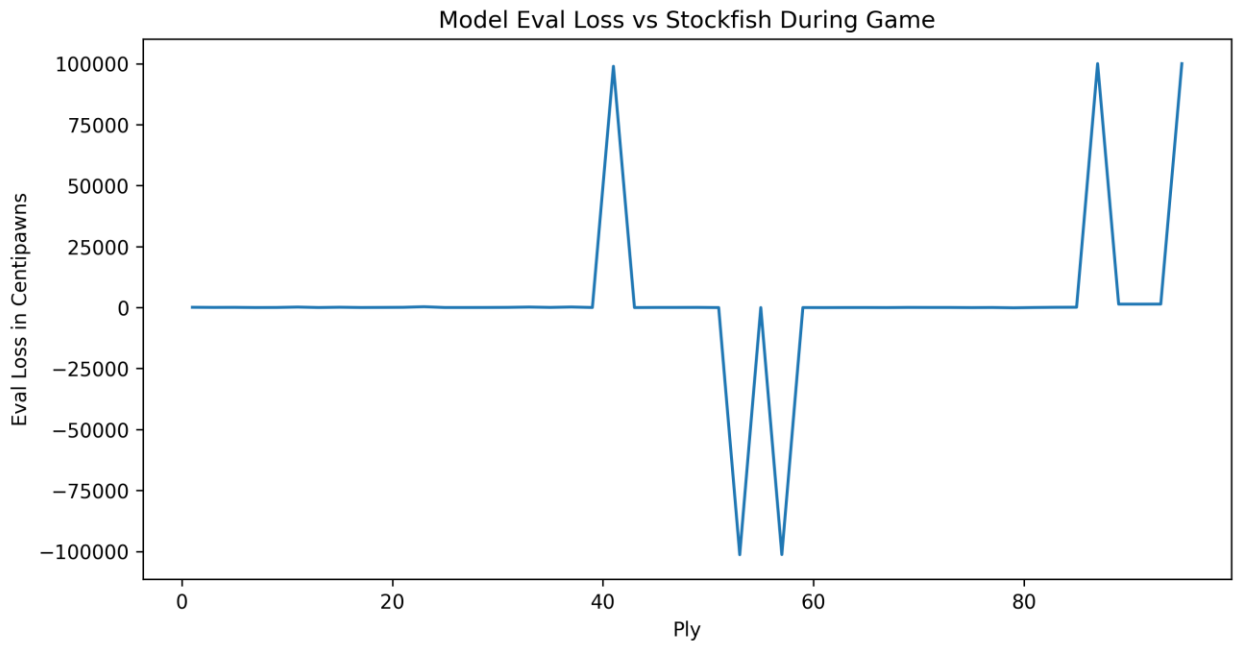
negamax

```
Final result: 1-0
Mean: 5654.222222222223
Median: 93.5
Trimmed mean: 136.2941176470588
90th percentile: 421.0
95th percentile: 25174.25
Max: 100102
Blunder rate >300cp: 0.19444444444444445
```



Minimax

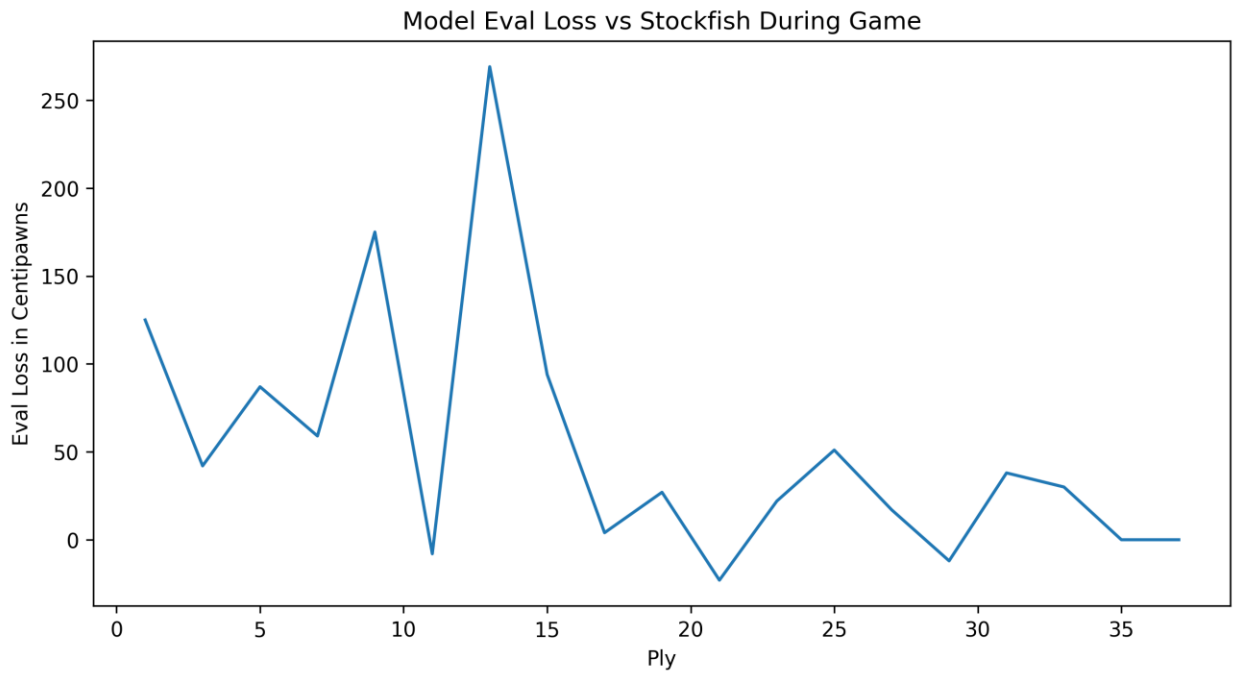
```
Final result: 1/2-1/2
Mean: 10583.041666666666
Median: 39.0
Trimmed mean: 6637.021739130435
90th percentile: 30656.300000000414
95th percentile: 100000.0
Max: 101372
Blunder rate >300cp: 0.1875
```



Depth 11

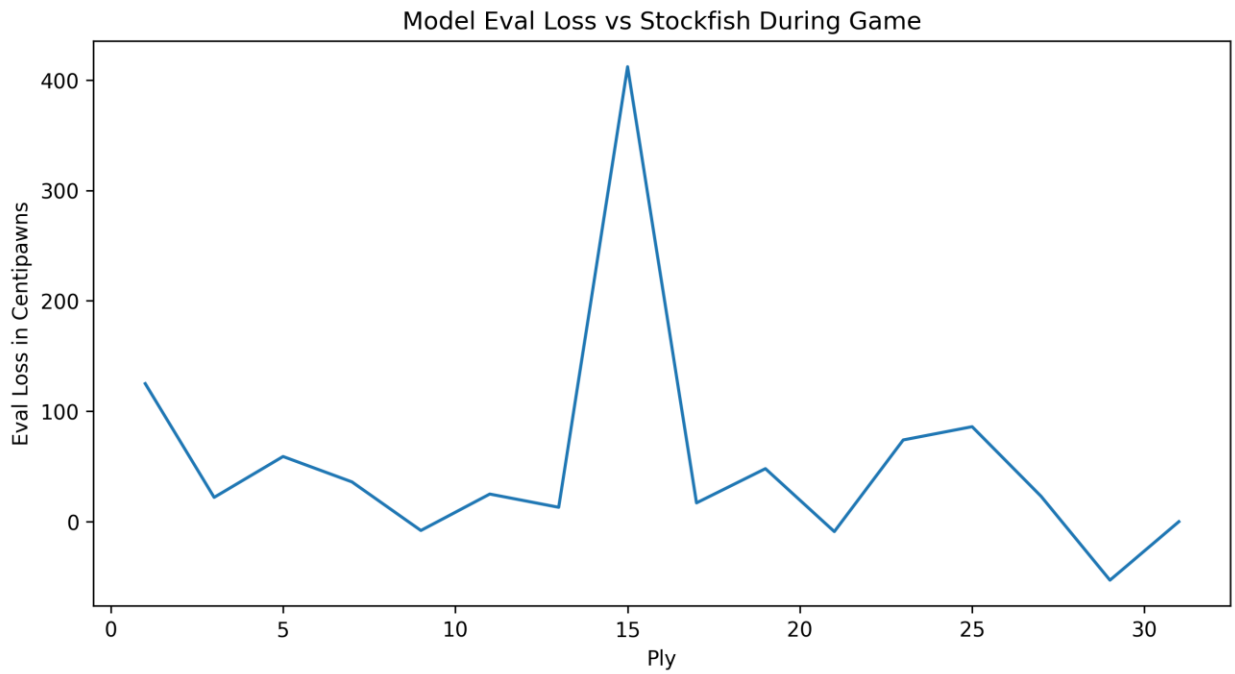
Negamax

```
Final result: 1-0
Mean: 57.0
Median: 30.0
Trimmed mean: 45.22222222222222
90th percentile: 134.99999999999997
95th percentile: 184.3999999999998
Max: 269
Blunder rate >300cp: 0.0
```



Minimax

```
Final result: 1-0
Mean: 63.125
Median: 30.5
Trimmed mean: 39.86666666666667
90th percentile: 105.5
95th percentile: 196.75
Max: 412
Blunder rate >300cp: 0.0625
```



Summary of CNN based chess model

The initial chess model performed moderately on first move evaluations although it had poor foresight into future moves. A minimax and negamax implementation were added to help search future moves with the model. In addition to minimax and negamax a material base heuristic was added to both minimax and negamax to prevent material-based blunders against the opponent. After comparing the different functions, the best performance was negamax due to its low rate of blunders even at higher depths. In conclusion, the model performs moderately well against beginner players, although it has a difficult time in the endgame section.